Adaptive Running Plan Generator

Product Requirements Document — Version 1 (Couch-to-5K)

---

1. Problem & Purpose

Existing couch-to-5K apps (e.g. the NHS app, Runna) are built around a fixed weekly template and, in Runna's case, a paid subscription. This project builds a personal, free alternative: a run-plan engine that generates a couch-to-5K schedule around the user's own available days, and adapts the plan when sessions are missed.

Primary purpose: a CV / portfolio project demonstrating algorithmic design (scheduling, constraint handling, plan adaptation) rather than CRUD or UI work alone.

Note: plan length is goal-specific. C25K runs 9 weeks; other distance goals (10K, 15K, half marathon, marathon), planned for later versions, will each have their own length — none of them are fixed to 9 weeks.

2. Audience

- Primary: the developer, as a personal training tool and a CV project.
- Secondary: anyone starting a couch-to-5K programme who wants a schedule that fits their own week rather than a fixed calendar.

3. Scope — Version 1

In scope:

- One goal only: **Couch-to-5K (C25K)** — a fixed 9-week run/walk interval progression.
- A plan generator that places sessions onto the user's chosen available weekdays, starting from a chosen start date.
- A replan function that adjusts the remaining schedule when sessions are missed (see Section 6).
- Automated tests (pytest) covering generation and replanning, including edge cases.
- Calendar export: a generated plan can be exported as an **.ics** file, importable into Google Calendar, Apple Calendar, or Outlook.

Out of scope for v1 (deferred):

- 10K, 15K, half marathon and marathon plans, and the recovery/speed/long session structure they require.
- Pace targets or pace-based speed work (effort-based cues only, e.g. "3 min hard").
- A front end / UI of any kind — v1 is a script or CLI producing an .ics file.
- Auto-detection of fitness level; the user simply selects C25K.
- Notes/journaling per session, and any calorie or step tracking.

3b. Tech Stack

- **Language:** Python (engine, tests, CLI). TypeScript may be used later if a web front end is built, but is out of scope for v1.
- **Testing:** pytest.

- **Calendar export:** the **ics** Python library (rather than hand-writing the .ics format), to keep focus on the scheduling logic.
- **Version control:** new GitHub repository, with a Python virtual environment (venv) for dependencies — consistent with previous projects.
- **Interface (v1):** command-line only — no front end. A UI (Figma design, then a web app) is deferred until after the engine and tests are complete; designing screens now would mean guessing at a UI for logic that doesn't exist yet.

4. Inputs & Outputs

Inputs (PlanRequest):

- **Goal** — fixed to C25K in v1.
- **Start date** — the date the plan begins.
- **Available days** — the weekdays the user can run (must supply at least 3, since C25K runs 3x/week).
- **Max session minutes** — used to sanity-check that a session fits the user's available time (C25K sessions are ~20–35 minutes including warm-up/cool-down).

Output (Plan):

- An ordered list of **Sessions**, one per scheduled run, each with: a date, a week number (1–9), a type (walk/run interval structure), a list of segments (e.g. "run 1 min, walk 90 sec, x8"), and a status (planned / done / missed).
- An .ics file representing the current plan, regenerated after any replan.

5. Core Rules (v1 — C25K)

- The plan always follows the standard 9-week C25K run/walk progression (fixed week-by-week structure; not user-configurable in v1).
- Exactly 3 sessions are scheduled per week, placed on the user's chosen available days.
- Sessions are spaced out across the week where possible, rather than clustered on consecutive days, to allow recovery.
- The plan is a pure function of its inputs: given the same request, it always produces the same schedule (no hidden state or randomness).

6. Missed-Session Handling (Replan)

Behaviour is modelled on how Runna handles this: a single missed session is not crammed in later, but persistent misses trigger a bigger adjustment (Runna's own "Plan Realignment" feature appears once more than three sessions, or a full week, have been missed).

Rules for v1:

- **Single missed session:** mark it "missed", then look at the remaining days in the current week and shift the rest of that week's sessions to make the best use of the days left. The total number of sessions that week may drop to 2 if there isn't a suitable day left, but the plan is not extended for a single miss.
- **Repeated misses:** if 3 or more sessions are missed in total, or a full week is missed, the current week is repeated rather than moving on — the user redoes that week's progression before advancing.
- The 9-week structure itself never changes; only which calendar dates each week's sessions fall on.

- Replanning only ever affects sessions that are still "planned" — completed sessions are never altered.

7. Acceptance Criteria

Each of these is intended to become a pytest test case:

- Given a start date and 3 available weekdays, **generate_plan** returns exactly 27 sessions (9 weeks x 3), each falling on one of the chosen weekdays.
- Given only 2 available weekdays, **validate_request** rejects the request (C25K needs 3 sessions/week).
- Marking one session as "missed" and calling **replan** reschedules only the remaining sessions in that week, onto the user's available days, without changing sessions in later weeks.
- Marking 3 sessions as "missed" within a rolling window triggers the current week to repeat, and all subsequent weeks shift forward by one week.
- A completed ("done") session is never modified by **replan**.
- The exported .ics file contains one calendar event per planned session, each with the correct date and a description of the run/walk structure.

8. Build Order

- **1. Data model** — PlanRequest, Session, Segment (done — see plan_model.py).
- **2. validate_request** — reject invalid requests (e.g. too few available days).
- **3. generate_plan** — build the fixed 9-week C25K schedule onto available days.
- **4. Tests** for 2 and 3, covering the acceptance criteria above.
- **5. replan** — single-miss shifting, then repeated-miss week restart, with tests.
- **6. .ics export** — convert a Plan into a calendar file.
- **7. CLI entry point** — collect inputs, run the engine, write the .ics file.

9. Definition of Done — Version 1

- generate_plan and replan are implemented and covered by passing pytest tests for all acceptance criteria in Section 7.
- A plan can be exported to a valid .ics file that opens correctly in at least one calendar app.
- Code is committed to GitHub with a README explaining the project, the rules above, and a disclaimer that the schedule is a heuristic, not medical or coaching advice.

Note: progression numbers and scheduling rules above are simplified for a first build and are not medical or coaching advice.